

Федеральное государственное автономное образовательное учреждение высшего
образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных технологий

Кафедра «Информатика и информационные технологии»

Направление подготовки/ специальность: 09.03.02 Информационные системы и технологии

ОТЧЕТ

по проектной практике

Студент: Федоров Г.И. Группа: 251-337

Место прохождения практики: Московский Политех, кафедра Информатика и
информационные технологии

Отчет принят с оценкой _____ Дата _____

Руководитель практики: Дагаев Александр Евгеньевич

Москва 2026

Оглавление

ВВЕДЕНИЕ	3
1. БАЗОВАЯ ЧАСТЬ ПРАКТИКИ.....	4
1.1 Настройка Git и репозитория	4
1.2 Документирование в Markdown.....	4
1.3 Создание статического веб-сайта	5
2. ВАРИАТИВНАЯ ЧАСТЬ ПРАКТИКИ.....	6
2.1 Постановка задачи	6
2.2 Архитектура базы данных	6
2.3 Тестирование	7
ЗАКЛЮЧЕНИЕ	9
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ	10
ПРИЛОЖЕНИЕ А.....	11
ПРИЛОЖЕНИЕ Б.....	11

ВВЕДЕНИЕ

Проектная практика является важным этапом обучения, направленным на закрепление теоретических знаний и приобретение практических навыков работы с современными инструментами разработки программного обеспечения.

В рамках практики были поставлены две основные цели: освоить базовые инструменты разработчика (система контроля версий Git, язык разметки Markdown, вёрстка веб-страниц) и реализовать собственный программный проект — файловую базу данных на языке C#.

Выбор темы вариативной части обусловлен тем, что базы данных являются фундаментальным компонентом большинства информационных систем.

Понимание того, как устроено хранение данных изнутри, помогает в дальнейшем эффективно работать с промышленными СУБД и принимать обоснованные архитектурные решения.

Практика состояла из базовой части, включавшей настройку Git-репозитория, документирование в Markdown и создание статического веб-сайта, и вариативной части, в ходе которой была спроектирована и реализована собственная файловая база данных с поддержкой CRUD-операций. Общая трудоёмкость составила 72 часа, работа выполнена индивидуально.

Результатом практики стал работающий программный продукт, техническая документация и веб-сайт с описанием проекта.

1. БАЗОВАЯ ЧАСТЬ ПРАКТИКИ

1.1 Настройка Git и репозитория

Создан личный репозиторий на платформе GitHub. Ссылка на репозиторий:

<https://github.com/GrigoriyFedorov/practice-2026-1>

Освоены основные команды Git:

- git clone — клонирование репозитория
- git add — добавление файлов в индекс
- git commit — фиксация изменений
- git push — отправка на удалённый сервер
- git checkout -b — создание новой ветки

Разработка велась с регулярной фиксацией изменений. Структура репозитория соответствует заданию: папки docs, reports, site, src, task.

1.2 Документирование в Markdown

Изучен синтаксис Markdown. Вся документация проекта оформлена с использованием:

- Заголовков разных уровней
- Нумерованных и маркированных списков
- Таблиц
- Блоков кода с подсветкой синтаксиса
- Изображений и ссылок

Итоговая документация размещена в файле docs/README.md и содержит: исследование предметной области, архитектуру базы данных, техническое руководство для начинающих, хронологию этапов работы.

1.3 Создание статического веб-сайта

Разработан статический сайт с использованием HTML5 и CSS3. Сайт содержит 5 страниц:

- Главная — аннотация проекта и скриншот меню
- О проекте — цели, UML-диаграмма классов, схема файловой структуры, листинг кода
- Участники — информация о разработчике и личный вклад
- Журнал — три записи о ходе работы со скриншотами
- Ресурсы — ссылки на документацию и полезные материалы

Сайт размещён в папке site/ репозитория. UML-диаграмма и схема архитектуры выполнены на чистом CSS без использования изображений. Ссылка на сайт:

<https://github.com/GrigoriyFedorov/practice-2026-1/tree/master/site>

2. ВАРИАТИВНАЯ ЧАСТЬ ПРАКТИКИ

2.1 Постановка задачи

Задача: разработать собственную файловую базу данных на C#, которая хранит объекты в формате JSON, обеспечивает типобезопасность через интерфейс IEntity и поддерживает базовые операции CRUD.

Для реализации выбраны:

- Язык: C# (.NET)
- Сериализация: System.Text.Json
- Хранение: отдельные JSON-файлы в папках-таблицах
- Идентификация записей: GUID

2.2 Архитектура базы данных

Разработана библиотека классов MyOwnDatabase, содержащая:

Интерфейс IEntity — определяет контракт для всех хранимых объектов:

```
public interface IEntity
{
    Guid Id { get; set; }
}
```

Класс DataBase — ядро базы данных, принимает путь к корневой папке.

Основные методы:

Save<T>(T entity) — создаёт или обновляет запись. Имя папки определяется по имени типа T. Если Id пустой, генерируется новый GUID. Объект сериализуется в JSON и сохраняется в файл {Id}.json:

```
string tablePath = Path.Combine(_rootPath,
    typeof(T).Name);
Directory.CreateDirectory(tablePath);

if (entity.Id == Guid.Empty)
    entity.Id = Guid.NewGuid();

string filePath = Path.Combine(tablePath,
    $"{entity.Id}.json");
string json = JsonSerializer.Serialize(entity);
File.WriteAllText(filePath, json);
```

`Load<T>(Guid id)` — загружает запись по идентификатору. Если файл не найден, возвращает `null`.

`LoadAll<T>()` — загружает все записи таблицы, читая все JSON-файлы из папки типа.

`Delete<T>(Guid id)` — удаляет файл записи по ID.

`DeleteAll<T>()` — удаляет всю папку таблицы с содержимым.

`Find<T>(Func<T, bool> predicate)` — загружает все записи и фильтрует их через LINQ по заданному условию - `return LoadAll<T>().Where(predicate).ToList();`

2.3 Тестирование

Для демонстрации работы базы данных разработано консольное приложение с интерактивным меню:

===== MENU =====

1. Добавить объект в БД
2. Найти объект по ID
3. Показать все объекты определенного типа
4. Найти объекты по условию
5. Удалить объект
6. Удалить все объекты определенного типа
0. Выход

=====

Приложение позволяет:

- Добавлять объекты классов User (поля: Name, Age, Email) и Product (поля: Title, Price, Category)
- Просматривать все объекты выбранного типа
- Искать по условию (например, пользователи старше 18 лет)
- Удалять отдельные записи и очищать таблицы полностью

Скриншоты работы программы представлены в Приложении А.

ЗАКЛЮЧЕНИЕ

В ходе проектной практики успешно выполнены все поставленные задачи:

Освоены инструменты разработки:

- Система контроля версий Git (clone, add, commit, push, branch)
- Язык разметки Markdown для оформления документации
- HTML и CSS для создания статических веб-сайтов

Разработана собственная файловая база данных на C#:

- Спроектирована архитектура (интерфейс IEntity, класс DataBase)
- Реализован полный набор CRUD-операций
- Добавлен поиск по условию через предикаты
- Создано консольное приложение для демонстрации

Подготовлена документация:

- Техническое руководство в формате Markdown
- Статический веб-сайт с диаграммами и скриншотами
- Итоговый отчёт по проектной практике

Проект демонстрирует понимание принципов хранения данных, сериализации и работы с файловой системой. Полученные знания могут быть применены при изучении более сложных систем управления базами данных.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Документация Microsoft по System.Text.Json: <https://learn.microsoft.com/ru-ru/dotnet/standard/serialization/system-text-json/how-to>
2. Руководство по Git: <https://git-scm.com/book/ru/v2>
3. Руководство по Markdown: <https://doka.guide/tools/markdown/>
4. Троелсен Э. Язык программирования C# 9 и платформа .NET 5. — М.: Вильямс, 2021.
5. Материалы репозитория build-your-own-x: <https://github.com/codecrafters-io/build-your-own-x>

ПРИЛОЖЕНИЕ А

Скриншоты работы консольного приложения:

```
=====MENU=====
```

1. Добавить объект в БД
2. Найти объект
3. Найти все объекты определенного типа
4. Найти объекты, соответствующие условию
5. Удалить объект
6. Удалить все объекты определенного типа
0. Выход

Выберите действие: |

```
Всего пользователей: 3
```

```
ID: 264189fc-d780-424f-8f13-3f5ec65b2b2b
```

```
Имя: Алиса
```

```
Возраст: 14
```

```
Email: alica123@mail.ru
```

```
ID: 45619e84-fcd3-42cd-a353-08858ab29b51
```

```
Имя: Григорий
```

```
Возраст: 22
```

```
Email: dddfreeerrr123@mail.ru
```

```
ID: f64ac094-19a0-400d-b690-c385d5427877
```

```
Имя: Петр
```

```
Возраст: 30
```

```
Email: qwerty@mail.ru
```

```
Введите минимальный возраст для поиска: 18
```

```
Найдено пользователей: 2
```

```
Григорий , возраст: 22, email: dddfreeerrr123@mail.ru
```

```
Петр, возраст: 30, email: qwerty@mail.ru
```

ПРИЛОЖЕНИЕ Б

Листинг кода

```

using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text.Json;

namespace MyOwnDatabase
{
    public class DataBase
    {
        private readonly string _rootPath;

        public DataBase(string rootPath)
        {
            _rootPath = rootPath;
            if (!Directory.Exists(_rootPath))
                Directory.CreateDirectory(_rootPath);
        }

        public void Save<T>(T entity) where T : IEntity
        {
            string tablePath = Path.Combine(_rootPath,
typeof(T).Name);
            Directory.CreateDirectory(tablePath);

            if (entity.Id == Guid.Empty)
                entity.Id = Guid.NewGuid();

            string filePath = Path.Combine(tablePath,
 $"{entity.Id}.json");
            string json =
JsonSerializer.Serialize(entity,
                new JsonSerializerOptions { WriteIndented
= true });
            File.WriteAllText(filePath, json);
        }

        public T Load<T>(Guid id) where T : IEntity
        {

```

```

        string filePath = Path.Combine(_rootPath,
typeof(T).Name, $"{id}.json");
        if (!File.Exists(filePath))
            return default;
        string json = File.ReadAllText(filePath);
        return JsonSerializer.Deserialize<T>(json);
    }

    public List<T> LoadAll<T>() where T : IEntity
    {
        string tablePath = Path.Combine(_rootPath,
typeof(T).Name);
        if (!Directory.Exists(tablePath))
            return new List<T>();
        var result = new List<T>();
        foreach (var file in
Directory.GetFiles(tablePath, "*.json"))
        {
            string json = File.ReadAllText(file);
            T entity =
JsonSerializer.Deserialize<T>(json);
            if (entity != null) result.Add(entity);
        }
        return result;
    }

    public void Delete<T>(Guid id) where T : IEntity
    {
        string filePath = Path.Combine(_rootPath,
typeof(T).Name, $"{id}.json");
        if (File.Exists(filePath))
            File.Delete(filePath);
    }

    public void DeleteAll<T>() where T : IEntity
    {
        string tablePath = Path.Combine(_rootPath,
typeof(T).Name);
        if (Directory.Exists(tablePath))

```

```
        Directory.Delete(tablePath, true);
    }

    public List<T> Find<T>(Func<T, bool> predicate)
where T : IEntity
    {
        return
LoadAll<T>().Where(predicate).ToList();
    }
}

public interface IEntity
{
    Guid Id { get; set; }
}
}
```